

Towards improved collaborative text editing CRDTs by using Natural Language Processing

Jim Bauwens
Software Languages Lab
Vrije Universiteit Brussel
Belgium
jim.bauwens@vub.be

Kevin De Porre
Software Languages Lab
Vrije Universiteit Brussel
Belgium
kevin.de.porre@vub.be

Elisa Gonzalez Boix
Software Languages Lab
Vrije Universiteit Brussel
Belgium
egonzale@vub.be

Abstract

Collaborative text editing systems are used in a variety of cloud-based products. To ensure that documents remain consistent between users, these systems often rely on CRDTs, operational transformation, or other techniques for achieving (strong) eventual consistency. CRDT-based approaches are appealing as they incorporate strategies to ensure that concurrent updates cannot conflict. However, these strategies do not necessarily take into account program semantics and may result in unexpected behaviour from the end-user's perspective.

For example, conflict resolution strategies in collaborative text editors may lead to duplicate words and incorrectly merged sentences. This position paper investigates the use of deterministic natural language processing (NLP) algorithms to improve the concurrency semantics of collaborative text editing systems that rely on CRDTs, aiming to provide a better end-user experience. We explore what is needed to ensure convergence, and highlight potential difficulties with the approach.

CCS Concepts: • **Computer systems organization** → **Embedded systems**; *Redundancy*; • **Networks** → Network reliability.

ACM Reference Format:

Jim Bauwens, Kevin De Porre, and Elisa Gonzalez Boix. 2023. Towards improved collaborative text editing CRDTs by using Natural Language Processing. In *10th Workshop on Principles and Practice of Consistency for Distributed Data (PaPoC '23)*, May 8, 2023, Rome, Italy. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3578358.3591330>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org. *PaPoC '23*, May 8, 2023, Rome, Italy

© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0086-6/23/05...\$15.00
<https://doi.org/10.1145/3578358.3591330>

1 Introduction

Conflict-free Replicated Data Types (CRDTs) are well-suited for cloud-based collaborative text editors, as they enable seamless collaboration without a central authority overseeing the updates [9, 10]. While CRDTs aim to prevent conflicts in concurrent updates, the conflict resolution strategies used by CRDTs may result in unexpected outcomes from the perspective of end-users of collaborative text editors. This is because CRDT implementations prioritise guaranteeing eventual consistency over preserving the document's logical structure. Typically, CRDTs avoid conflicts based on arbitrary (but deterministic) logic, e.g. a last-writer-wins set [10] breaks ties between concurrent writes by ordering them through replica identifiers. Users expect a seamless and intuitive experience when working on shared documents, but current approaches can result in a document that does not reflect their intended changes, leading to duplicated words or incorrectly merged sentences.

In this paper, we propose enhancing the end-user experience of collaborative text-editing software by means of natural language processing (NLP) techniques [4]. NLP is an interdisciplinary field that combines linguistics, computer science, and machine learning to develop algorithms and models that can perform tasks such as text classification, sentiment analysis, machine translation, and question-answering. NLP already plays a crucial role in many applications, such as chatbots, speech recognition, and text-to-speech systems, allowing machines to understand and process human language in a more human-like manner [11]. We explore the design of a CRDT that uses hints of NLP to steer the process that merges concurrent/conflicting operations when editing a document. In contrast to using arbitrary rules when reordering concurrent operations, we argue that the application could use NLP to find orderings that lead to a more grammatically correct state, resulting in a document that reads more smoothly and is free of grammatical errors. The main challenge is to design a CRDT that is efficient and ensures that the different replicas deterministically solve conflicts based on grammatical rules.

2 Concurrency semantics

As mentioned before, CRDTs typically ensure convergence using conflict resolution strategies that encode the data

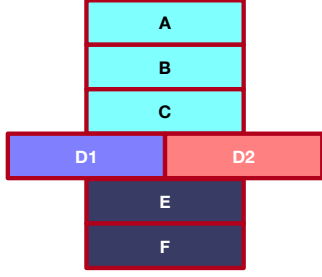


Figure 1. Sequence of operations applied in a replicated system. D1 and D2 are concurrent updates.

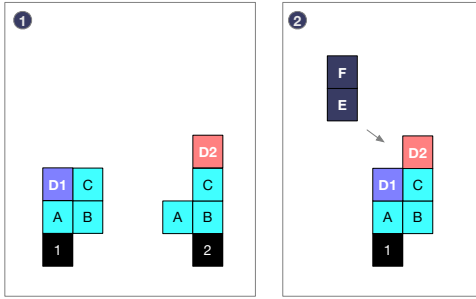


Figure 2. Representation of the operations and the clocks for two replicas, showing two concurrent operations D1 and D2, and finally the arrival of two new operations on replica 1. The graphical representation is based on the visualization techniques by C. Baquero [1].

type's concurrency semantics. These semantics dictate how multiple, concurrent updates to a replica are resolved, such that the data structure remains consistent and converges to a single state across all replicas, even in the presence of network partitions. To solve conflicting updates, the CRDT implementation relies on an arbitrary ordering of concurrent updates.

For example, consider the LWW-register (Last Writer Wins Register) CRDT. In an LWW-register a single value can be stored. When two updates are concurrent, the update with the greatest 'timestamp' is considered to be the "winner", and will take precedence over the other update. Typically, this timestamp can be a (non-logical) timestamp, sometimes combined with the network ID of a replica to avoid the case where two timestamps may be equal. The winning update is the one that all replicas will eventually converge to and is replicated across all replicas.

Figure 1 shows a graph that visualises the sequence of operations applied to a replicated CRDT system (as seen in Figure 2). Operations A, B, and C follow each other sequentially, they have a direct causal relation. Then, two concurrent operations are applied on two replicas: D1 and D2. Finally, operations E and F are applied, and they causally follow all previous operations.

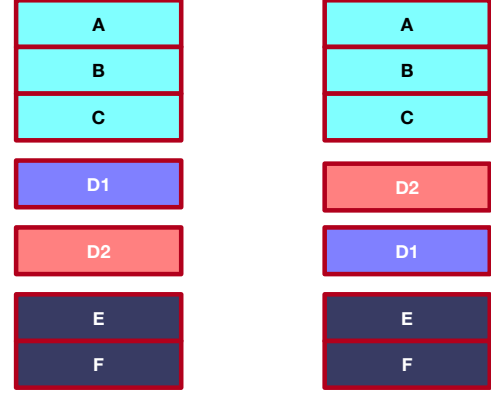


Figure 3

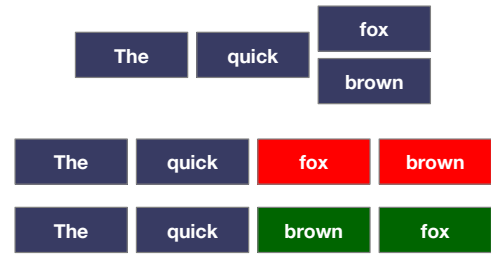


Figure 4. Example orderings for two concurrent inserts. Only one order is grammatically correct.

Figure 3 visualises the two different orderings in which these operations can be applied. Depending on the wanted semantics, CRDT implementors will decide on a particular set of rules for deciding the ordering of concurrent operations. To ensure convergence, the rules must guarantee that the result of the ordering picked is equal across all replicas.

Several CRDT algorithms exist for collaborative text editing [5, 7, 8, 13]. These algorithms guarantee convergence but may lead to anomalies from the end-user's perspective. Depending on the granularity adopted by the algorithm, words can be merged on a character level Kleppmann et al. [6], or the order of words can become grammatically incorrect. For example, Figure 4 shows the possible outcomes when concurrently inserting two words at the same location in a document. Depending on the replica identifier, the sentence might converge to *The quick brown fox* or *The quick fox brown*. In the next section, we explore how we might tackle this problem by guiding convergence through grammatical information.

3 Approach

Natural Language Processing (NLP) [4, 11, 12] is a branch of artificial intelligence that focuses on the interactions between computers and humans using natural language. NLP algorithms can be used to understand the semantic meaning

of words, sentences, and documents, and to analyze their grammatical structure.

In this work, we propose to incorporate NLP algorithms in the conflict resolution strategy of collaborative text editing CRDTs so as to produce more grammatically correct and intuitive results upon concurrent updates. NLP algorithms can be used to understand the semantic meaning of words, sentences, and documents, and to analyze their grammatical structure and dependencies. This information can then be used to guide the process of merging concurrent updates in a way that produces a document that is grammatically correct and easy to read.

In this section, we explore three approaches to how one could use information from an NLP system to improve the concurrency semantics of collaborative text editors.

3.1 Naive approach: re-application

The causal history of operations on a CRDT can be encoded in the form of a directed acyclic graph (DAG). Such a causal graph represents the relationships between the different operations, indicating which operations occurred before others. The current state of a CRDT can be seen as a particular topological ordering of its causal graph. Typically CRDT algorithms ensure that this ordering is always the same on every node, ensuring a consistent result and convergence of all replicas without synchronization.

To incorporate NLP into the concurrency semantics of a CRDT, we can directly encode the causal history of a CRDT as a causal DAG, and try to influence the ordering of operations. We can search for all permutations of valid topological orderings and pick an ordering which is the most grammatically correct. Using Grammatical Error Correction (GEC) NLP algorithms [12], it is possible to obtain a score for a sentence, and as such, we can pick the highest-ranking permutation.

To ensure that the causal graph does not grow unboundedly and that an extremely high number of permutations aren't generated, the size of the data stored in the CRDT needs to be limited. This can be done by determining the causal stability of operations[2] and only generating permutations for operations that are not yet causally stable.

This first approach has two main drawbacks:

- For every operations that arrives, the topological order of operations needs to be recomputed. This means that it is not possible to have monotonic reads; e.g. words might suddenly get re-ordered again when new updates arrive.
- Moreover, all permutations need to be ranked with an NLP algorithm on the arrival of any operations. This might impact the speed and performance of the application.

3.2 Ahead-of-time ranking

To avoid unneeded re-computations, we present a second approach in which modified sentences are ranked ahead of time (as in, given a correctness score based on their grammatical structure), at the source replicas. This information can then be included with the payload of operations that are being propagated to other replicas. If two concurrent operations are mutually exclusive (each cancels out the other operation), the included ranking meta-data can be used on receiving replicas to immediately decide the order in which they should be applied. Of course, this is very dependent on the operations applied, and newly arriving operations may conflict with the strategy and require a fallback to the previous approach in order to ensure convergence. To improve this strategy, we can use the same ranking information before operations are propagated. Some local updates might become redundant by newer updates (for example, when a sentence is being rewritten), and as such, we can filter out redundant operations.

3.3 Understanding grammatical dependencies

Instead of only looking at ranking information between different orderings of concurrent inserts, our third proposal is to use information from grammatical dependencies. For example, in the sentence "*A brown fox*", *brown* is an adjective, and it has a clear dependency on the noun *fox* (as visualised in figure 5). Now imagine that concurrently on one replica the phrase *brown fox* was removed, but on the other replica the sentence was modified to *A quick brown fox*. Figure 6 visualises the applied operations on the sentence. *quick* is an adjective for *fox*, and as *fox* is being removed, it does not make sense to keep one but remove the other. We believe that either both the noun and adjectives are removed, or the newly inserted adjective with the noun should be kept. Figure 7 shows the traditional outcome for common sequential text-editing CRDTs, and the two possible aforementioned outcomes if NLP meta-data would be used for determining word dependencies. In all cases, even though *The* has a grammatical dependency on *fox*, we cannot remove it as that would violate the intent of the user. *brown* will always be removed, as its removal will not create any broken semantics (it depends on *fox*, but *fox* does not depend on it).

In light of this, we propose to keep track of grammatical dependencies and evaluate the meaning of these dependencies when inserting or removing content.

4 Discussion

Even if CRDTs are aided by using NLP techniques, it will never be possible to fully understand the user's intent. Because of this, the system *will* make errors at times. It might be interesting to expose optional merging options to the front end of collaborative text editors so that it is possible to choose alternative sentences, i.e., reifying merging decisions to the

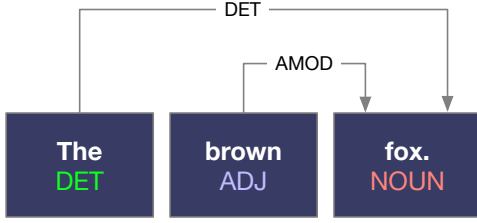


Figure 5. Example of grammatical dependencies between words. The adjective 'brown' has an adjectival modifier (amod) relationship with the noun 'fox'. 'The' has a determiner (DET) relationship with the noun.

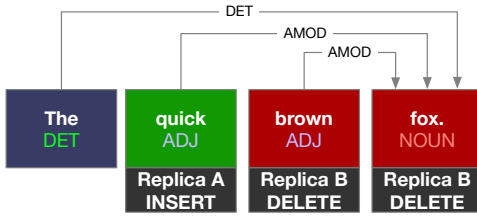


Figure 6. Example of two concurrent operations that have shared grammatical dependencies.

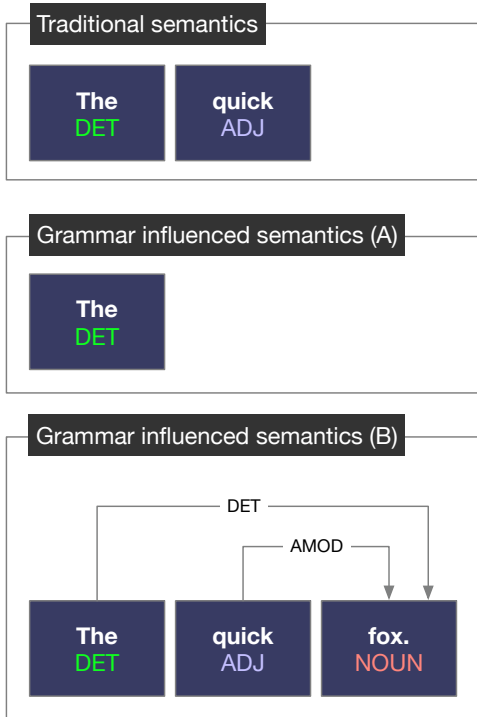


Figure 7. Possible resulting states for the applied operations in the previous figure.

end-user. How exactly this could be implemented remains to be seen, as it should not hamper a user's workflow, or cause additional complexity (what if two users concurrently decide on a different strategy). Depending on the situation, it might also be possible to require synchronisation for a decision such as this.

An alternative to traditional GEC NLP algorithms [12] is to use content-aware AI systems. GEC NLP algorithms are not able to extract content and intent from a text. By using more powerful techniques, for example with the use of GPT-3 models [3], it may be possible to find better-merging strategies that take the entire textual context into account. Currently, however, these techniques demand extreme computational power and would hamper the reactivity if a replica is made dependent on them.

5 Conclusion

In conclusion, this paper investigated the potential of using deterministic NLP algorithms to enhance the concurrency semantics of collaborative text editing systems that rely on CRDTs. Current CRDT algorithms prioritize eventual consistency over preserving the document's logical structure, leading to issues such as duplicated words or incorrect sentence merges. By incorporating NLP techniques, we propose that CRDTs use hints of NLP to steer the process of merging concurrent operations when editing a document. This approach would prioritize preserving the document's logical structure, resulting in a document that reads more smoothly and is free of grammatical errors, providing a more intuitive and seamless user experience. This is a promising avenue for future research, as it could greatly improve the end-user's experience of cloud-based collaborative text editors.

References

- [1] Carlos Baquero. 2020. Causality is Graphically Simple. <https://doi.org/10.48550/ARXIV.2012.09086>
- [2] C. Baquero, P. S. Almeida, and A. Shoker. 2017. Pure Operation-Based Replicated Data Types. *CoRR* abs/1710.04469 (2017). arXiv:1710.04469
- [3] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. <https://doi.org/10.48550/ARXIV.2005.14165>
- [4] Michael A. Hedderich, Lukas Lange, Heike Adel, Jannik Strötgen, and Dietrich Klakow. 2021. A Survey on Recent Approaches for Natural Language Processing in Low-Resource Scenarios. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, Online, 2545–2568. <https://doi.org/10.18653/v1/2021.naacl-main.201>
- [5] R. Hyun-Gul, J. Myeongjae, K. Jin-Soo, and L. Joonwon. 2011. Replicated abstract data types: Building blocks for collaborative applications. *J. Parallel and Distrib. Comput.* 71, 3 (2011), 354 – 368.

- [6] Martin Kleppmann, Victor B. F. Gomes, Dominic P. Mulligan, and Alastair R. Beresford. 2019. Interleaving Anomalies in Collaborative Text Editors. In *6th Workshop on Principles and Practice of Consistency for Distributed Data (PaPoC 2019)*. ACM, Article 6. <https://doi.org/10.1145/3301419.3323972>
- [7] Brice Nédelec, Pascal Molli, Achour Mostefaoui, and Emmanuel Desmontils. 2013. LSEQ: An Adaptive Structure for Sequences in Distributed Collaborative Editing. In *Proceedings of the 2013 ACM Symposium on Document Engineering (Florence, Italy) (DocEng '13)*. Association for Computing Machinery, New York, NY, USA, 37–46. <https://doi.org/10.1145/2494266.2494278>
- [8] Nuno Preguica, Joan Manuel Marques, Marc Shapiro, and Mihai Letia. 2009. A Commutative Replicated Data Type for Cooperative Editing. In *2009 29th IEEE International Conference on Distributed Computing Systems*. 395–403. <https://doi.org/10.1109/ICDCS.2009.20>
- [9] N. Preguiça. 2018. Conflict-free Replicated Data Types: An Overview. arXiv:1806.10254 [cs.DC]
- [10] M. Shapiro, N Preguiça, C. Baquero, and M. Zawirski. 2011. *A comprehensive study of Convergent and Commutative Replicated Data Types*. Technical Report 7506. INRIA.
- [11] Amirsina Torfi, Rouzbeh A. Shirvani, Yaser Keneshloo, Nader Tavaf, and Edward A. Fox. 2020. Natural Language Processing Advancements By Deep Learning: A Survey. *CoRR* abs/2003.01200 (2020). arXiv:2003.01200 <https://arxiv.org/abs/2003.01200>
- [12] Yu Wang, Yuelin Wang, Kai Dang, Jie Liu, and Zhuo Liu. 2021. A Comprehensive Survey of Grammatical Error Correction. *ACM Trans. Intell. Syst. Technol.* 12, 5, Article 65 (dec 2021), 51 pages. <https://doi.org/10.1145/3474840>
- [13] Stephane Weiss, Pascal Urso, and Pascal Molli. 2010. Logoot-Undo: Distributed Collaborative Editing System on P2P Networks. *IEEE Trans. on Parallel and Distributed Systems* 21, 8 (Aug. 2010), 1162–1174.